ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

XÂY DỰNG VÀ TINH CHỈNH PHIÊN BẢN TỐI GIẢN CỦA MÔ HÌNH LLAMA 2

BÁO CÁO BÀI TẬP

Học phần: AIT3009# 1

Sinh viên thực hiện

Nguyễn Thị Thanh Huyền - 23020381

Nguyễn Thị Minh Ly - 23020399

Đặng Minh Nguyệt - 23020407

Giảng viên hướng dẫn

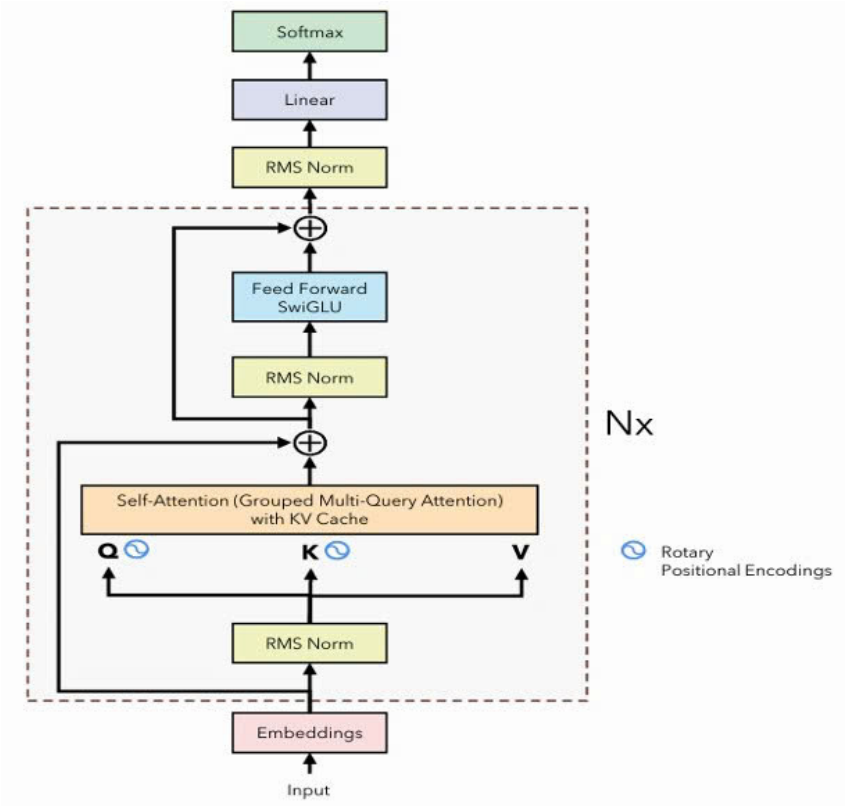
ThS. Ngô Minh Hương

HÀ NỘI - 2026

1 Kiến trúc LLAMA-2

1.1 Kiến trúc tổng quát

Llama 2 là mô hình ngôn ngữ lớn dựa trên kiến trúc Transformer dạng decoder-only. Mô hình nhận chuỗi token đầu vào, xử lý qua các lớp Transformer decoder với cơ chế causal self-attention, và dự đoán token tiếp theo thông qua lớp tuyến tính kết hợp softmax.
Mỗi decoder layer tuân theo thiết kế pre-norm, với luồng xử lý: RMSNorm → Causal Self-Attention → Residual connection, tiếp theo là RMSNorm → Feed-Forward Network → Residual connection. Đặc biệt, cơ chế Rotary Positional Embeddings (RoPE) được tích hợp trực tiếp vào các vector query và key để mã hóa linh hoạt thông tin vị trí tương đối.



Hình 1: Kiến trúc Llama 2

1.2 Cài đặt

Cài đặt các thành phần chính của Llama-2 trong `llama.py`, `rope.py`, `optimizer.py`, `classifier.py` chi tiết như sau:

- `llama.Attention.forward`: Thực hiện self-attention bằng cách biến đổi đầu vào thành Q, K, V và tổ chức theo nhiều head để học các quan hệ khác nhau. Thông tin vị trí được tích hợp qua RoPE, trong khi GQA cho phép chia sẻ K và V giữa các head nhằm tối ưu bộ nhớ. Kết quả được tổng hợp và chiếu qua lớp tuyến tính để tạo đầu ra.

- `llama.RMSNorm.norm`: Thực hiện chuẩn hóa RMSNorm dựa trên giá trị trung bình bình phương của vector đầu vào, kết hợp với ϵ để đảm bảo ổn định số học. Phép chuẩn hóa này điều chỉnh độ lớn của vector nhưng vẫn giữ nguyên hướng trong không gian biểu diễn.

- `llama.Llama.forward`: Kết hợp embedding, các decoder layer và RMSNorm để xây dựng biểu diễn của chuỗi. Biểu diễn này được chiếu sang không gian từ vựng để tạo logits; khi suy luận chỉ tính tại vị trí cuối nhằm tối ưu hiệu năng.

- `llama.Llama.generate`: Sinh văn bản theo cơ chế tự hồi quy, trong đó chuỗi hiện tại được đưa trở lại mô hình để dự đoán token tiếp theo với temperature sampling. Chuỗi được giới hạn theo độ dài tối đa và các token được nối dần để tạo kết quả.

- `rope.apply_rotary_emb`: Áp dụng RoPE bằng cách biểu diễn Q và K dưới dạng cặp và thực hiện phép quay phụ thuộc vị trí thông qua các hàm sin–cos, từ đó giúp tích hợp thông tin vị trí trực tiếp vào attention và bảo toàn quan hệ tương đối giữa các token.

- `optimizer.AdamW.step`: Cập nhật tham số bằng cách duy trì trung bình động bậc nhất và bậc hai của gradient, kèm hiệu chỉnh bias để ổn định quá trình học. Tham số được cập nhật với gradient đã chuẩn hóa, sau đó áp dụng weight decay tách biệt.

- `classifier.LlamaEmbeddingClassifier.forward`: Sử dụng biểu diễn ẩn tại token cuối làm đặc trưng đại diện cho chuỗi, sau đó được điều chỉnh qua dropout, đưa vào lớp phân loại và log-softmax để thu được phân phối xác suất trên các lớp.

```
PS C:\23020407\ASM1-Development-LLM-model> .\.venv\Scripts\python.exe rope_test.py
Rotary embedding test passed!
PS C:\23020407\ASM1-Development-LLM-model> .\.venv\Scripts\python.exe optimizer_test.py
Optimizer test passed!
PS C:\23020407\ASM1-Development-LLM-model> .\.venv\Scripts\python.exe sanity_check.py
Your Llama implementation is correct!
```

Hình 2: Kết quả chạy 03 test cài đặt

2 Đánh giá mô hình

2.1 Sinh phần tiếp nối văn bản

Yêu cầu: Bắt đầu với câu: *"I have wanted to see this thriller for a while, and it didn't disappoint. Keanu Reeves, playing the hero John Wick, is"*.

Nhận xét:

- Văn bản sinh ra xuất hiện nhiều câu vô nghĩa, không liên quan đến ngữ cảnh ban đầu. Nguyên nhân do mô hình được pretrained trên tập dữ liệu truyện thiếu nhi, dẫn đến văn phong kể chuyện đơn giản, phi logic so với bối cảnh phim hành động.

```
Temperature is 0.0
I have wanted to see this thriller for a while, and it didn't disappoint. Keanu Reeves, playing the hero John Wick, is this day. He was playing
with his toy car, driving it around the living room. Suddenly, he heard a loud crash. He had broken the car and was very sad.
John was angry and he shouted at his little brother. He was only three years old and he was only three. He was only three years old. He was very
 ups
---------------
Wrote generated sentence to generated-sentence-temp-0.txt.
load model from stories42M.pt
Temperature is 1.0
I have wanted to see this thriller for a while, and it didn't disappoint. Keanu Reeves, playing the hero John Wick, is coming a new friend to pl
ay with."
Iwriting brought a mask, but I wore plain white beats. The sun shone brightly. I be a happy, obedient lung.
It'scious to be a true hero. What should I play?"here Ialen Once upon a time, there was a jolly youth named Lucy.
---------------
Wrote generated sentence to generated-sentence-temp-1.txt.
```

Hình 3: Kết quả sinh văn bản với temperature = 0 và temperature = 1

- Khi *temperature = 0*, mô hình có xu hướng chọn các từ có xác suất cao nhất, nên câu văn ít đa dạng hơn nhưng vẫn bị lặp và thiếu tự nhiên. Khi *temperature = 1*, mức độ ngẫu nhiên tăng lên, giúp đa dạng hóa từ vựng nhưng cũng làm giảm tính mạch lạc, khiến các câu rời rạc, khó hiểu.

2.2 Zero-shot Prompting

Zero-shot prompting là phương pháp yêu cầu mô hình thực hiện một tác vụ mà không cung cấp bất kỳ ví dụ minh họa nào, chỉ dựa trên mô tả (prompt) của bài toán. Đối với tác vụ này, nhóm tiến hành điều chỉnh (tune) tham số *batch size* với nhiều giá trị khác nhau nhằm đánh giá ảnh hưởng đến hiệu năng.

Dữ liệu	**Giá trị thử nghiệm**	**Giá trị tối ưu**	**Mặc định**		**Tối ưu**	
			dev-acc	**test-acc**	**dev-acc**	**test-acc**
SST-5	10, 20, 25, 30	25	0.213	0.224	**0.223**	**0.237**
CFIMDB	1, 5, 10, 15	1	0.502	0.213	**0.584**	0.623

Bảng 1: Kết quả tối ưu batch size cho zero-shot prompting (10 là giá trị mặc định)

Lưu ý: File `data/cfimdb-test.txt` không chứa nhãn thật (gold-label). Do đó, độ chính xác trên tập test của CFIMDB không được sử dụng để điều chỉnh tham số.

```
load 8544 data from data/sst-train.txt
load 1101 data from data/sst-dev.txt
load 2210 data from data/sst-test.txt
eval: 100%|[redacted]| 45/45 [00:04<00:00,  9.06it/s]
eval: 100%|[redacted]| 89/89 [00:09<00:00,  9.10it/s]
dev acc :: 0.223
test acc :: 0.237
```

```
load 1707 data from data/cfimdb-train.txt
load 245 data from data/cfimdb-dev.txt
load 488 data from data/cfimdb-test.txt
eval: 100%|[redacted]| 245/245 [00:07<00:00, 34.98it/s]
eval: 100%|[redacted]| 488/488 [00:24<00:00, 20.09it/s]
dev acc :: 0.584
test acc :: 0.623
```

Hình 4: Kết quả chạy zero-shot prompting với tham số tối ưu trên SST-5 và CFIMDB

Nhận xét:

- Việc điều chỉnh tham số là cần thiết. So với cấu hình mặc định, tuning giúp cải thiện hiệu năng trên cả hai tập dữ liệu, đặc biệt với CFIMDB.
- Sự khác biệt về *batch size* tối ưu xuất phát từ đặc trưng của từng bộ dữ liệu. SST-5 hưởng lợi từ *batch size* lớn hơn vì dữ liệu ngắn và cần gradient ổn định, còn CFIMDB hưởng lợi từ *batch size* nhỏ vì review dài và chứa nhiều sắc thái ngữ nghĩa riêng biệt.

2.3 Fine-tuning

Fine-tuning được kỳ vọng cải thiện hiệu năng so với zero-shot promptin trên tác vụ phân loại. Trong thực nghiệm này, các tham số gồm *learning rate*, số *epochs*, *dropout* và *batch size* được tối ưu tuần tự: mỗi lần chỉ thay đổi một tham số, cố định các tham số còn lại, sau đó chuyển sang tham số tiếp theo.

Dữ liệu	*lr*	*epochs*	*batch size*	*dropout*
SST-5	1e-5, **2e-5**, 3e-5, 4e-5, 5e-5	**5**, 8, 10	64, **80**, 100	0.1, **0.3**, 0.5
CFIMDB	1e-5, **2e-5**, 3e-5, 4e-5, 5e-5	**5**, 8, 10	8, **10**, 16	0.1, **0.3**, 0.5

Bảng 2: Các tham số được thử nghiệm trong fine-tuning (**giá trị mặc định** được in đậm, giá trị tối ưu được gạch chân)

Dữ liệu	**Mặc định**		**Tối ưu**	
	dev-acc	**test-acc**	**dev-acc**	**test-acc**
SST-5	0.425	0.445	0.425	0.445
CFIMDB	0.865	0.457	**0.886**	0.488

Bảng 3: Kết quả trước và sau khi tối ưu tham số trong fine-tuning

```
load model from finetune-5-2e-05.pt
load 1101 data from data/sst-dev.txt
load 2210 data from data/sst-test.txt
eval: 100%|[redacted]| 14/14 [00:12<00:00,  1.16it/s]
eval: 100%|[redacted]| 28/28 [00:25<00:00,  1.12it/s]
dev acc :: 0.425
test acc :: 0.445
```

```
load model from finetune-5-4e-05.pt
load 245 data from data/cfimdb-dev.txt
load 488 data from data/cfimdb-test.txt
eval: 100%|[redacted]| 25/25 [00:14<00:00,  1.74it/s]
eval: 100%|[redacted]| 49/49 [00:27<00:00,  1.78it/s]
dev acc :: 0.886
test acc :: 0.488
```

Hình 5: Kết quả chạy fine-tuning với tham số tối ưu trên SST-5 và CFIMDB

Nhận xét: Fine-tuning cải thiện rõ rệt so với zero-shot prompting, cho thấy tầm quan trọng của việc điều chỉnh mô hình theo dữ liệu đặc thù của bài toán.